

Training Guide

Fit the label-free parameters from telemetry and emit a deployable network —
no fault history required

A copy-paste walkthrough of the training half: what the trainer fits, how to run it, how to bring your own historian data, how to verify the result, and exactly where training hands off to deployment. Python is used here for initial fitting only; the runtime model is Java.

Document	Model Training Guide
Product	Jneopallium Self-Supervised Maintenance Guardian
Trainer	scripts/demo-self-supervised-maintenance/ train_ss_maintenance_model.py
Labels used	0 (self-supervised)
Output	Deployable Jneopallium bundle (descriptor + context + layers)
Author	Dmytro Rakovskyi — Kharkiv, Ukraine
Date	2 July 2026
License	BSD 3-Clause

Contents

1. What you will achieve
2. Where training ends and deployment begins
3. Prerequisites
4. Step 1 — run the trainer
5. Step 2 — what it fitted (the parameters)
6. Step 3 — the bundle it produced
7. Step 4 — bring your own data
8. Step 5 — verify with the tests
9. Step 6 — the hand-off to deployment
10. Retraining and drift
11. Troubleshooting
12. Appendix — training cheat-sheet

1 What you will achieve

By the end you will have a set of fitted, label-free parameters — per-regime standardisation, cross-sensor reconstruction weights, and health-baseline percentiles — written into a complete, deployable Jneopallium bundle that the Java runtime boots from directly. No failure labels are used at any point.

The safety ceiling applies to training too

Nothing the trainer produces can actuate a device. The emitted model is advisory by construction (SsMaintConfig forbids disabling it), and severity is calibrated against healthy history — never against labelled failures, because there are none.

2 Where training ends and deployment begins

Phase	What happens	Document
Training (this guide)	Fit standardisation, cross-sensor weights, baselines; emit the bundle	Training Guide
Deployment	Load the bundle, wire telemetry + feedback, run the advisory scorer	Deployment Guide

The hand-off is one directory: `worker/src/main/resources/model/self-supervised-maintenance/`. Training **writes** it; deployment **reads** it.

3 Prerequisites

- Python 3.9+ (standard library only — no pip install needed).
- A JDK 17 toolchain and Maven if you also want to run the Java runtime tests.
- The repository checked out; commands below are run from the repo root.

4 Step 1 — run the trainer

```
cd scripts/demo-self-supervised-maintenance
python train_ss_maintenance_model.py
```

Expected output (values are deterministic for the shipped synthetic corpus):

```
Self-Supervised Maintenance – initial (label-free) training

Trained on 6 assets x 2000 ticks, labels used: 0
Sensors                : 8
Health baseline mean   : 0.7781
Health baseline p99    : 2.0808
Health baseline p999   : 2.7544
Neurons in bundle      : 4

Deployable bundle -> worker/src/main/resources/model/self-supervised-maintenance
```

`run_all.sh` (or `run_all.ps1`) runs the trainer and then the Python tests in one go.

5 Step 2 — what it fitted (the parameters)

All three parameter groups are self-supervised — fitted from telemetry alone, on a trusted-healthy window:

- **Per-regime standardisation** — the mean and standard deviation of every sensor in every operating regime, so a reading is judged relative to its load.
- **Cross-sensor reconstruction weights** — for each sensor, a ridge-regression model that predicts it from the other sensors. This is the label-free core: the target of each fit is another sensor, never a failure label.
- **Health-baseline percentiles** — the mean, p95, p99 and p999 of the reconstruction error over the healthy window, used to calibrate severity in the asset's own units.

Why ridge regression

A small ridge (L2) penalty keeps the cross-sensor models stable when sensors are correlated — which, on a coupled skid, they always are. The solver is a plain normal-equation solve in pure Python; no third-party linear-algebra package is required.

6 Step 3 — the bundle it produced

File	Purpose
model-descriptor.json	Manifest: layers, signals, data sources, labelFree=true, labelsUsed=0
production-context.json	IContext for the Entry runner (neuron classes, inputs, aggregator)
layer-0-input.json	Telemetry ingest descriptor + sensor order
layer-1-selfsupervisedhealth.json	Reconstruction neuron + fitted weights & standardisation
layer-2-fusion.json	Hypothesis neuron + baseline percentiles + fusion constants
layer-3-onlinelearning.json	Feedback-adaptation neuron + bounds/rate-limit/freeze
layer-4-advisorygate.json	Read-only gate + initial thresholds + dedup window
fitted-model.json	Raw fitted parameters for audit / reference

The layer files carry the parameters as neuron fields, so the files that record your fit are the files the runtime binds. This mirrors the industrial model's bundle layout.

7 Step 4 — bring your own data

For a real site, replace the synthetic generator with your historian export. The trainer reads a list of per-tick rows through `synth_telemetry.generate`; point it at your data instead. Each canonical row must have:

- The same **sensor names** the model expects (or edit `SENSORS` to match your tag set).
- An integer **regime** column (operating point / load band). If you don't have one, derive it from a load or power band.

- **No labels.** The training window should be **mostly healthy** — this is the one assumption that matters. If an asset was already faulty during the window, exclude that period so it does not poison 'normal.'

Fleet baselines for cold starts

If an asset has too little history to define its own normal, fit against sibling assets in the same regime. The model reports domain-shift and uncertainty so an unfamiliar asset reads as 'watching' rather than making confident claims.

8 Step 5 — verify with the tests

```
cd scripts/demo-self-supervised-maintenance
python -m unittest tests.test_ss_maintenance -v
```

The Python tests confirm the trainer produced a sound model:

- baseline percentiles are correctly ordered (mean < p95 < p99 < p999);
- all cross-sensor weights are finite;
- healthy frames rarely exceed the asset's own p99 (nuisance rate under 5%);
- **every injected fault family is separated from the healthy baseline without labels** (reconstruction error above 2× p999), and the dominant residual points at the right sensor group;
- the bundle is written and valid (labelFree=true, labelsUsed=0, safetyMode=ADVISORY).

The runtime behaviour — evidence accumulation, feedback adaptation, the gate — is validated separately by the Java suite `SelfSupervisedMaintenanceModuleTest` (14 tests). Run it with `mvn -pl worker -Dtest=SelfSupervisedMaintenanceModuleTest test`.

9 Step 6 — the hand-off to deployment

Training is done when the bundle directory is written and the tests pass. The Deployment Guide takes it from there: it loads the same bundle with the Entry runner, wires your OPC UA / MQTT telemetry to `AssetTelemetrySignal` and your operator feedback to `OperatorFeedbackSignal`, and starts producing advisories.

10 Retraining and drift

Refit the baselines when the plant genuinely changes (new product, rebuild, seasonal shift) — not on every blip. Day-to-day adaptation is handled at runtime by the feedback loop, which needs no retraining and no redeploy. A practical cadence: refit the standardisation and baselines on a rolling healthy window monthly or after a known change, and let the online loop handle the rest.

11 Troubleshooting

Symptom	Likely cause	Fix
Healthy assets flag often	Training window wasn't healthy	Exclude faulty periods; refit
A fault isn't separated	Sensor coupling too weak / missing channel	Add the coupled channel

Everything reads uncertain	Data unlike training (domain shift)	Refit on representative data
Percentiles look wrong	Too few ticks per regime	Give each regime enough samples

12 Appendix — training cheat-sheet

```
# fit + emit bundle
python scripts/demo-self-supervised-maintenance/train_ss_maintenance_model.py

# python tests (trainer + label-free separation)
python -m unittest tests.test_ss_maintenance

# java runtime tests
mvn -pl worker -Dtest=SelfSupervisedMaintenanceModuleTest test
```

Training here means fitting 'normal' from telemetry and emitting a deployable network — with zero labels. Everything about a specific failure is learned later, at runtime, from operator feedback.